Lance

Executive summary | Wittgenstein AI Tournament

Local, evidence-grounded security review for Python

> Lance turns a small Python file into a short review queue. A deterministic parser finds a few high-risk code patterns and records the evidence, an optional local model explains each one, and a person makes every decision.

The problem

Manual triage of risky code is slow. Asking a general-purpose language model to "find vulnerabilities" tends to produce noisy findings not tied to anything in the code, and usually means sending proprietary source to a hosted service. Reviewers need a short, checkable list.

What Lance does

- **Evidence first.** A tree-sitter parser reads one Python file, finds calls to a fixed set of risky functions and records closed-vocabulary facts about each, such as an f-string built from a request value.
- **Bounded reachability.** Within the same file, it checks whether the call is in a top-level entry point (a route or main-block function) or a function called directly by one. Callers that are not entries, such as a two-hop chain, leave the candidate with reach "unknown".
- **Optional local explanation.** A locally hosted model returns a verdict, summary and suggested fix for each open candidate. The mechanical report works without it.
- **Human decision.** Candidates start unreviewed; only an explicit accept or reject by a person changes that. Lance never runs, edits or patches the code under review.

Three supported pattern families

Lance is not a general vulnerability detector; it recognises only these limited patterns:

Family	Flagged as a candidate	Discharged by the extractor
Shell invocation	`os.system`, `os.popen`, or `subprocess` with `shell=True` or a non-list command	`subprocess` with a list and without `shell=True`
Constructed SQL	`execute` or `executemany` on text built by concatenation, f-string, format or percent	query with a parameter list or tuple
Dynamic execution and unsafe deserialization	`eval`, `exec` or `pickle.loads` with direct string-construction expressions or recognised request-variable tracing; `yaml.load` with a positional input. Bare parameters and keyword-only inputs are not covered. Subject to reachability/discharge rules.	`yaml.safe_load`

What the labels mean

A **candidate** is a hypothesis: a pattern the extractor could not rule out, with evidence for a human to check. A **discharged** item means one of the extractor's own rules matched, such as a safe call form, or a function with no recognised entry status and no detected direct callers within the file. It is not proof that the program is safe, or that the code is unreachable from other modules. The model sees only the candidate record and at most three source lines: limited context, not a guarantee against model mistakes.

Who it is for, and what is new

Lance suits security engineers reviewing small Python files who want an offline mechanical first pass and an optional explanation. Its contribution is structured evidence from deterministic rules, kept separate from model reasoning and the human decision; it is not a taint proof or guard-dominance analysis.

Architecture, privacy and efficiency

The explanation step uses **Qwen3-Coder-30B-A3B-Instruct** (about 30B parameters in total, about 3B active per token), quantised to Q4_K_M, a file of roughly 18 GB served by llama.cpp under the alias qwen-coder-local. Its published weights are Apache-2.0; that says nothing about Lance itself, whose repository has no licence file. By default requests go to a loopback address, so no cloud inference is required. The endpoint is configurable, so this describes the default setup, not a guarantee of network isolation or a privacy certification. Run folders contain a snapshot of the reviewed source and the raw model response, unredacted, and users must protect them.

For efficiency, Lance skips the model for discharged items, calls it once per open candidate, and needs no training. No energy, carbon or memory measurements have been made, so none is claimed.

Human gate and robustness

- A model reply is accepted only if it is a JSON object whose verdict is exactly exploitable, smell or insufficient, which cites only the given candidate id, and which passes a small phrase denylist (payloads, proofs of concept, curl commands).
- Output that fails these checks is recorded as refused. A timeout, connection or HTTP failure is recorded as unavailable. In both cases the item stays unreviewed and the mechanical report is unaffected.
- "Cited" means the reply passed format and citation checks. It does not mean the claim was verified or that the code is exploitable. There is no automatic acceptance and no automatic patching.
- Each attempt stores a diagnostic category and timing, with no exception text. The raw response is kept as received. The default wait is 15 seconds per call and is configurable.

Evidence today

Evidence	Count	What it shows
Automated tests	80 passing	Extractor expectations, one-hop and unknown reachability, human decisions, citation checks and refusals, timeout and HTTP errors, sanitised diagnostics. Re-run 7 Nov 2026 on this snapshot.
Curated fixtures	7 files	Three flagged, three clean, and one helper reached from a route; expected outcomes are pinned by tests.
Live local call	1	One candidate from a flagged fixture; 21.5 s with a 120 s budget; the model's verdict (exploitable) passed the format and citation checks; status stayed unreviewed until a person rejected it as a test fixture; source and run snapshots were unchanged.

This is a smoke test, not a benchmark. No precision, recall or accuracy figure exists, and Lance has not been run on large or unseen repositories.

Limitations

Lance reads one file at a time and considers only top-level functions, with one hop of reachability. It does not follow calls across files or into class methods, and its source tracing is limited. Unknowns need manual inspection. Heuristic discharges and phrase checks are not guarantees. The test data is small. The local model can be slow or fail, which is why the mechanical report stands on its own.

Next three months (planned, not built)

- Grow a labelled evaluation set, including unseen and benign edge cases, and report measured results.
- Follow Python calls across files, with unknowns stated explicitly.
- Add one predicate family: an unchecked return value from a fallible call.
- Only later, explore a C grammar that emits the same human-first evidence format.